26

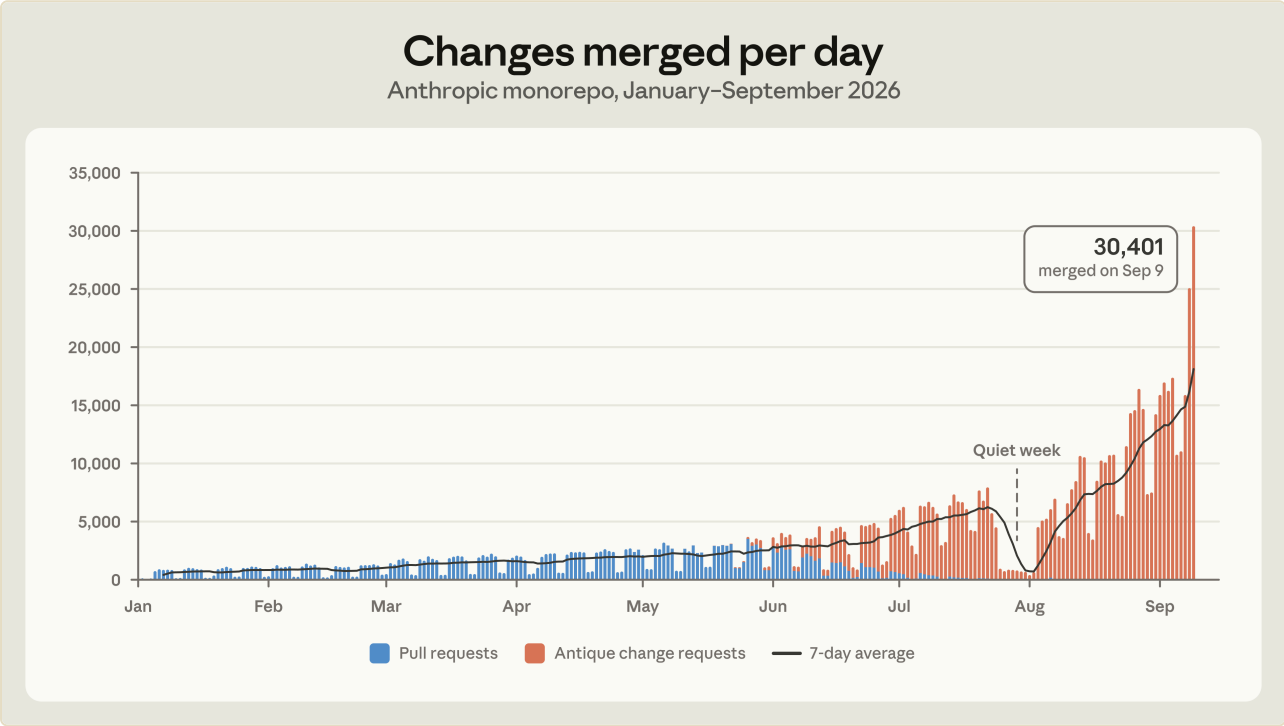
Anthropicにおけるソースコード管理の再設計

— Redesigning source control at Anthropic —

公開日	2026-10-13
原題	Redesigning source control at Anthropic
著者	Anthropic Engineering Team
原文	https://www.anthropic.com/engineering/redesigning-source-control
翻訳	Claude（機械翻訳／Anthropic）
編集	2026-10-14

Anthropicでは、コーディングはもはやシステム構築や研究のボトルネックではありません。Claudeは研究ループを自律的に反復し、複雑なプログラムをマイクロ最適化し、これまでに見たことのない速度とスケールで新しいサービスをゼロから構築できるようになりました。今やボトルネックとなっているのは、人間のスケールのために作られた開発者インフラ全体、つまりソースコード管理、ビルド、CIです。

2026年5月時点で、Anthropicでは1日あたり平均約2,500件のコード変更がマージされていました。9月時点では、1日あたり30,000件以上をマージしています。この成長速度が近いうちに鈍化するとは考えていません。コードをマージしたいという潜在的な需要は実質的に無限であり、ボトルネックを1つ取り除くたびに、生産するコードの量がさらに増加するのです。



2026年1月から9月までの、モノレポへの1日あたりのマージ変更数。

この成長を支えるために、私たちはScionという独自のバージョン管理システムを構築し、わずか5ヶ月でモノレポをgitから完全に移行しました。同じ期間に、コードレビュー画面（Antiqueと呼ばれています）、コード検索、リッチなソースコード管理用IAMシステム、大量のレビュー済みコードを管理・マージするための新しい手法なども構築し、リリースしました。この記事では、特にScionをどのように構築したかに焦点を当てます。これは興味深いケーススタディになります。Claudeのコードを書く能力こそが、そもそも私たちをこの状況に追い込んだ要因であり、*同時に*、この状況に対処するための高品質でスケーラブルなシステムを迅速に構築できた理由でもあるからです。

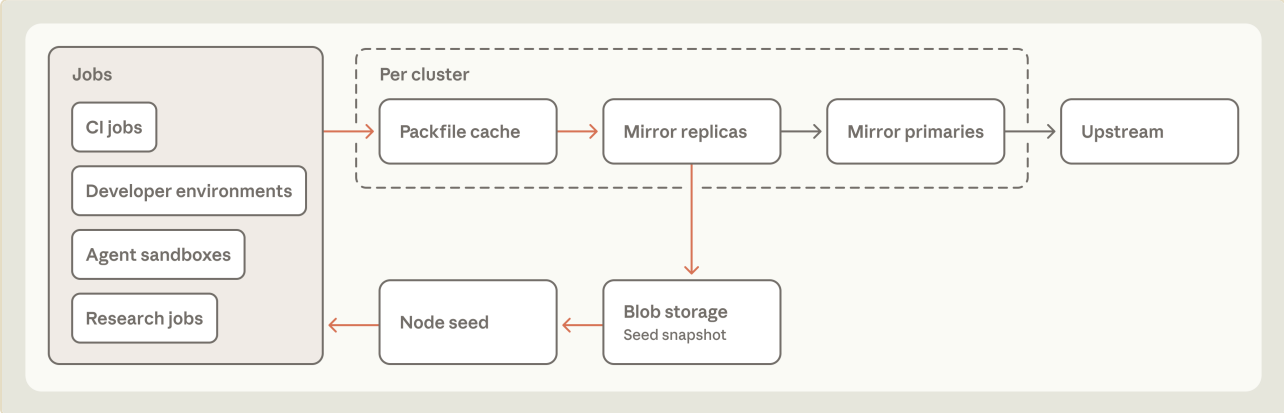
モノレポの成長

私たちが構築したものを掘り下げる前に、自分たちのワークロードがいかに特異なものであるかを認めておく必要があります。驚くことではありませんが、Anthropicはほぼどの企業よりもソフトウェアを書くためにClaudeを使っているため、人間のスケール向けに作られたツールがどこで限界を迎えるかを、私たちが最初に目にすることが多いのです。ソースコード管理はその一例に過ぎません。近い将来、多くの組織がこうした問題に直面するとは想定していませんが、AIコーディングツールを使う他の組織も、そう遠くないうちに同じ圧力点を経験するだろうと予想しています。だからこそ、私たちが構築したものと学んだことを共有したいと考えました。

gitは分散型システムです。すべてのクローンは完全かつ自己完結的なリポジトリであり、差分の計算、履歴やblameの読み取り、任意のリビジョンへのチェックアウトを完全にオフラインで行うことができます。大多数のリポジトリにとって、これは正しい設計であり、gitが標準として定着した大きな理由の一つです。

私たちのモノレポが成長するにつれて、この設計空間にますます収まらなくなってきました。mainブランチの履歴だけで約100GBあり、*圧縮済みの*作業コピーでさえ25GBを超えます。CI用だけで1日あたり2,500万回程度のコードチェックアウトを行っており、開発者の環境、エージェントのサンドボックス、研究ジョブはこれに含まれていません。これを単純にフェッチすれば、1日あたり600PBを超えるgitトラフィックになってしまいます。

gitには、リポジトリの一部だけを扱うためのツール、例えばシャロークローン、スパースチェックアウト、プロミサーリモートなどが用意されています。これらに加えて、巨大なモノレポのための標準的な回避策、例えば内部のgitミラーやblobストレージに保存されたリポジトリのスナップショット（シード）もあります。私たちはこれらすべてを使ってきました。ピーク時には、負荷に対処するために数百台の内部gitミラーを運用していました。これは時間を稼いでくれましたが、マージ率が増加するにつれて運用が難しくなっていきました。シードはリフレッシュの間にmainの最先端からどんどん取り残され、差分やblameのような履歴に依存する操作を部分クローン上で動かし続けるには、ますます細心の注意が必要になりました。



gitチェックアウトをスケールさせる典型的なセットアップ。

一歩引いて考えてみると、私たちは多くの企業と同様に、1つの正となるコピーを持つ中央集権的なシステムとしてgitを使っている一方で、そのクライアントのほぼすべてが、定義上オンラインであるエージェントやジョブであることに気づきました。gitを分散的なオフライン作業に優れたものにしている特性が、リポジトリの成長に合わせてスケールする私たちの能力を、むしろ損なっていたのです。そこで私たちは自問しました。「エージェントが常にオンラインであるという前提を最初から念頭に置いてソースコード管理を設計するなら、どのようなものになるだろうか？」と。

エージェントスケールのソースコード管理の原則

エージェントによるコーディングが普及する中で、ソフトウェア開発のあり方に関する多くの一般的な前提が変化しています。私たちはScionを構築する際にこれらの変化を受け入れ、いくつかの核心的な原則においてgitとは根本的に異なるアプローチを取ることができました。巨大なモノレポのためのソースコード管理には先行事例があり、特にGoogleのPiperやMetaのSaplingが有名です。私たちは彼らが苦労して得た教訓を踏まえてScionを構築することができました。私たちの原則は以下の通りです。

クライアントは常にオンラインである。 gitは、クライアントがオフラインで動作し、リポジトリの完全なコピーを必要とすることを前提としています。私たちのクライアントのほぼすべてがエージェントやジョブであることから、私たちはScionのクライアントとプロトコルを、常に到達可能なサーバーを前提に設計し、クライアントは必要なときにデータをフェッチするようにしました。

クライアントとサーバーの非対称性。 gitでは、すべてのクライアントが本格的なサーバーになる能力を持っています。Scionでは、ファイルの履歴やblameに関する問いに答えたり、コミットを作成したり、マージやリベースを実行したりするのはサーバーです。クライアントは、サイズに上限のあるローカルオブジェクトキャッシュを維持し、作業コピーのファイルをディスク上に配置し、ファイルシステムの変更を監視し、ユーザーがサーバーと使いやすい形でやり取りできるようにします。クライアント自体はほとんど計算を行いませんが、その代わり、ローカルに必要なデータははるかに少なくて済みます。

包括的なソースコード管理サービスは、多くの新しいアプリケーションの基盤となる。 gitでは、ある行を最後に変更したのは誰かや、2つのコミットがどのように関連しているかといった、リポジトリに関する問いに答える必要のあるCIジョブやツールは、リポジトリのローカルコピーを保持するか、ベンダー固有のAPIが答えを提供してくれることを期待するしかありません。Scionでは、クライアントが使うソースコード管理の問い合わせはすべてAPIとしても公開されているため、多くの場合、ジョブはローカルリポジトリをまったく必要としません。私たちの自動bisectパイプラインやrevertパイプラインのような社内ツールの多くは、この安価でプログラムから利用できるソースコード管理へのアクセスがなければ、大規模には実用的ではなかったでしょう。

すべてが水平方向にスケールしなければならない。ピーク時には数百台のgitミラーを運用していましたが、これはキャパシティを大きく圧迫し、リポジトリがミラーのディスクやメモリの限界を超えた時点でスケールしなくなりました。Scionでは、リポジトリのデータは分散されており、すべてのコンポーネントが独立してスケール可能です。サーバー上であっても、リポジトリの完全に実体化されたコピーはどこにも存在しません。packfile（gitの圧縮されたオブジェクトのバンドル）に相当するものもないため、リポジトリのメンテナンスも不要です。

コミットは安価で、インクリメンタルで、ロックフリーでなければならない。gitでは、ローカルでコミットしてから、自分の作業を共有するためにプッシュします。Scionでは、コミット、リベース、マージはすべてサーバー上で行われるため、プッシュするものは何もありませんが、これらの操作はそれぞれ高速である必要があります。コミットを作成し、それに連なるデータ構造を更新するコストは、リポジトリ全体のサイズではなく、変更のサイズに比例してスケールしなければなりません。そうでなければサーバーが処理に追いつけなくなります。Scionの開発ライフサイクルでは、ref（ブランチやタグなど、コミットへのポインタ）の使用を、mainのrefやリリースブランチを除いては推奨していません。なぜなら、すべてのサーバーが同意しなければならない*グローバル*な状態は、スケーリング上の問題を引き起こす可能性が高いからです。

作業コピーはオプションであり、設計上スパースである。私たちのモノレポは非常に大きく、CIのために1日に数千万回もコードをチェックアウトするため、できる限りチェックアウトを避けたいと考えました。Scionでは、例えばあるコミットとそのマージベースとの差分が必要なCIジョブは、サーバーに問い合わせることができます。書き込みについても同様で、多くの書き込みワークロードもチェックアウトを省略し、完全にAPI経由で機能させることができます。Scionのプロトコルは、クライアントが要求したディレクトリのみを含むスパースチェックアウトと、必要に応じてファイルを実体化する仮想化チェックアウトを中心に設計されているため、ジョブがディスク上にファイルを必要とする場合でも、クライアントは実際に読み取った分だけのコストしか払いません。

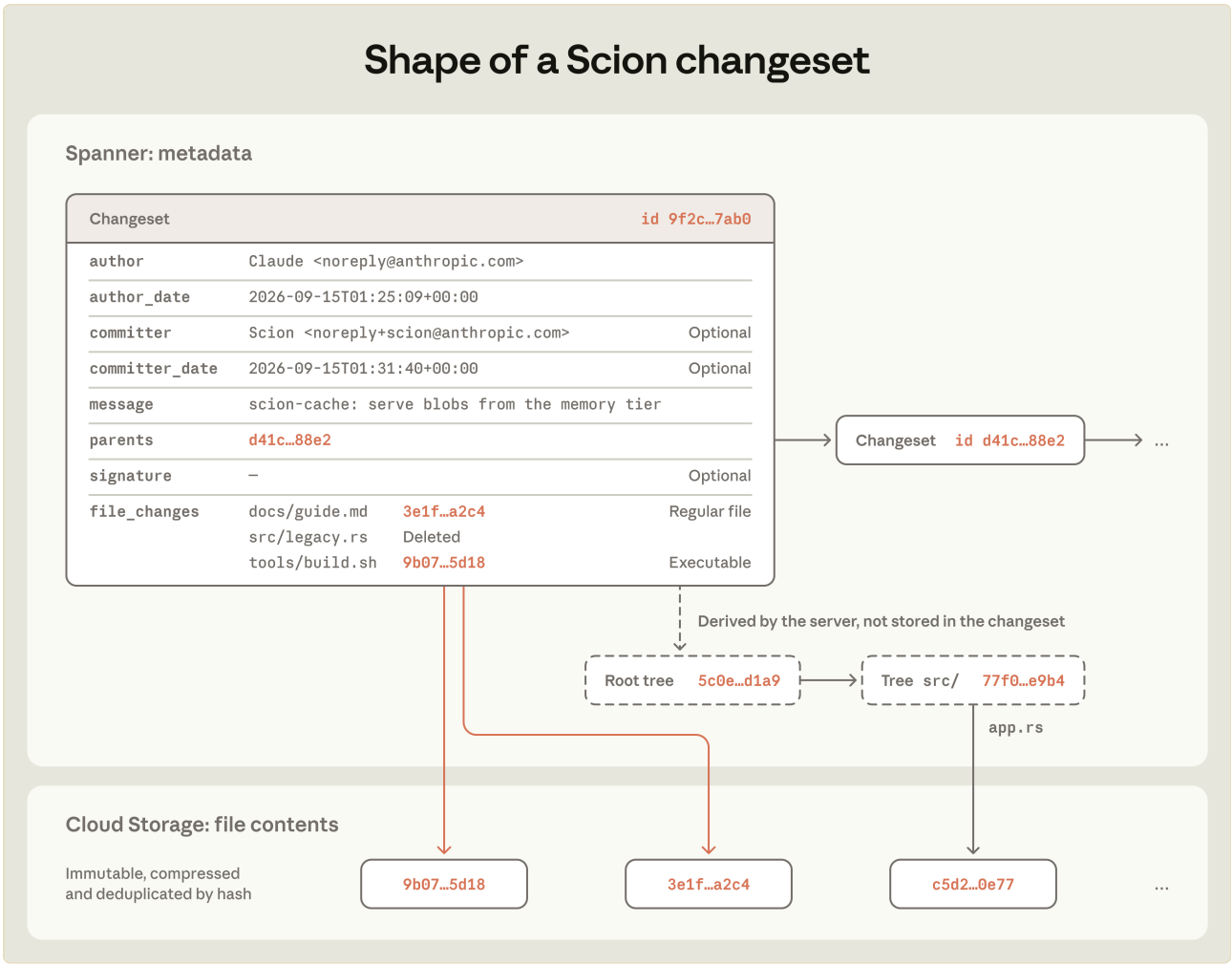
Scionの仕組み

これらの原則がScionの構築において実際にどのように組み合わさったかを理解しておくと役に立ちます。ここで得られた知見は、他のソースコード管理システムや、より広くソフトウェア開発ライフサイクルの他の段階にも応用できるはずです。

データ構造

Scionでは、作業の単位は「チェンジセット」であり、write-ahead logの1エントリです。チェンジセットは、gitのコミットから期待されるものの大部分を保持しています。しかしgitのコミットがツリーへのポインタを保持するのに対し、チェンジセットは、最初の親から見て変更があったファイルのリスト1と、blobストレージ内の圧縮済

みファイル内容へのコンテンツアドレス方式のポインタを保持します。チェンジセットはサーバー側で作成され、作成された瞬間からどこでも利用可能になります。



Scionにおけるチェンジセットの形。

この構造のおかげで、受け入れなければならないpackfileもなく、ロックやサーバー間の調整もなく、動かすべきrefもないため、非常に高いスループットでチェンジセットを受け入れることができます。Scionは技術的にはrefをサポートしていますが、コミットの作成もフェッチもrefを伴わないため、refを移動させる操作は、チェンジセットの作成とは別に行われる、Spannerの単一行に対するcompare-and-swapにすぎません。

チェンジセットは変更内容のみを記録するため、それ単体では、ある行を最後に編集したのは誰か、あるいはどのコミットがあるファイルに影響を与えたかといった問いに即座には答えられません。こうした問いに答えるために、Scionはチェンジセットから追加のデータ構造を構築します。ファイルのblameやパスの履歴専用の構造、コミットグラフ用のスキップリストなどです。チェンジセット自体の直接的な一部ではないものすべてを、私たちは「派生データ」と呼び、1つの汎用的な仕組みを通じてすべて計算します。各種類の派生データは再帰的に定義されており、あるチェンジセットに対する値は、そのチェンジセットの親に対する値から計算されます。

この仕組みには3つの利点があります。第一に、ある操作を高速化するような新しいデータ構造が必要になったとき、新しい種類の派生データを簡単に追加できます。第二に、種類ごとに、計算を即座に行うか、作業を遅延させるかを選ぶことができるため、必要のないデータを実体化する手間を避けられます。第三に、クライアントはこれらのデータ構造の内部を一切見ないため、クライアントを変更せずにこれらを最適化できます。システムが成長する中で、私たちはコピー検出やパスベースのファイル検索のための新しいインデックスなど、新しい種類の派生データを何度も追加してきました。

Scionのスケーリング

少数のScionサーバー群が、チェンジセットの取得、差分の計算、マージベースの検索、blobのフェッチなど、クライアントが必要とするあらゆる操作のためのソースコード管理APIを提供します。すべてのソースコード管理データはコンテンツアドレス方式であるため、あるデータは一度保存されたら変化せず、どのサーバーも他のサーバーに確認を取らずにそれをキャッシュできます。その結果、サーバーは水平方向にスケール可能で、必要な調整もほとんどありません。サーバーは本質的には巨大なキャッシュであり、その場で派生データを計算し、結果をSpannerに書き戻すこともできます。

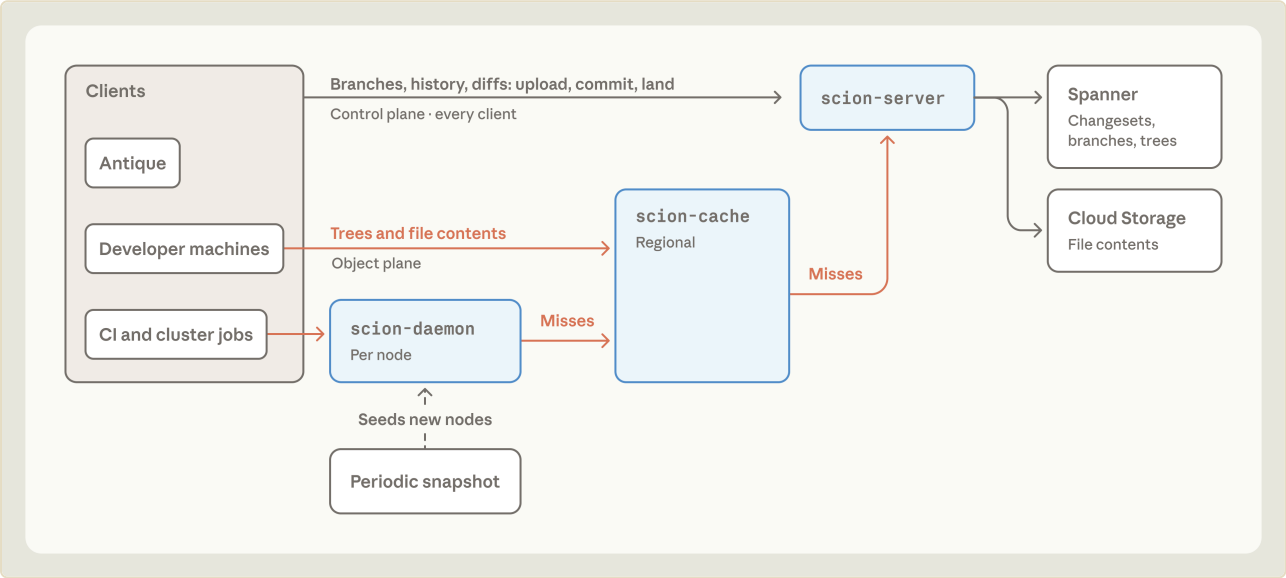
Scionの多くの利用者は、もはや自前のローカルリポジトリを維持する必要がありません。以前は、特にコミットの祖先関係やblameについてソースコード管理に問い合わせる必要があるCIジョブやツールにとって、リポジトリをフェッチして維持することが大きな保守負担でした。移行の初期、まだ独自のクライアント、リポジトリフォーマット、プロトコルを持っていなかった頃は、Scionはこうした「メタデータ」の問い合わせにのみ答えていました。ファイルの内容をフェッチし、作業コピーを実体化することは、はるかにデータ量の多い操作であり、それ自体のための設計が必要でした。

Scionのscnクライアントは、gitのフェッチプロトコルから重要な点でいくつか異なっており、その結果、gitサーバーよりも大幅に少ないCPUしか必要としない、高度にスケーラブルなアーキテクチャに適しています。Scionのネゴシエーションは、クライアントが既に何を持っているかをサーバーに伝えるものではなく、クライアントが欲しいツリーとblobのみを対象として動作します。クライアントはルートツリーから出発し、ローカルストアに対して欠けているツリーを確認し、それらを再帰的にリクエストし、同じプロセスをblobに対しても繰り返します。すべてのリクエストは決して変化しない特定のオブジェクトを対象としているため、どのキャッシュでもそれに答えることができ、このアーキテクチャは従来の階層型キャッシュによく似ています。このプロセスは、単一の巨大なpackfileとして返されるレスポンスが存在しないため、効率的で再開可能でもあります。スパースチェックアウトや仮想化チェックアウトについても特別なサポートは必要ありません。クライアントは単に欲しくないツリーをリクエストしないだけだからです。

このサーバー群の中で、クライアントはソケット経由でscion-daemonと呼ばれるデーモンと通信し、このデーモンがノード全体のオブジェクトを保存します。私たちのノードの大半はモノレポへのアクセスを必要としており、同じノード上のジョブはほとんど同じオブジェクトを再利用します。各デーモンは定期的に更新されるスナップショットから起動し、その後はそのリージョン内のツリーおよびblobキャッシュサーバー、scion-cacheと

のみ通信します。これらのキャッシュサーバーは、デーモンが消費するディスクフォーマットのまま、オブジェクトをメモリから直接ストリーミングするため、クライアントの欠けているオブジェクトを計算したり、デルタ圧縮を行ったりする時間を費やす必要がありません。キャッシュにオブジェクトがない場合、それは権威あるscion-serversに問い合わせ、scion-serversが必要に応じてSpannerやクラウドストレージから読み取ります。

scion-daemon、scion-cache、scion-serverはいずれも、このプロトコルについて同一のRPCに応答するため、クライアントはキャッシュのトポロジーを知る必要がありません。理論上はどの層も他のどの層とも通信できますが、障害ドメインを分離するために、私たちは階層構造を厳格に強制しています。あるリージョンのscion-cacheで発生した障害が上流のscion-serversを圧倒し、システム全体をダウンさせてしまう、という事態は避けたいからです。



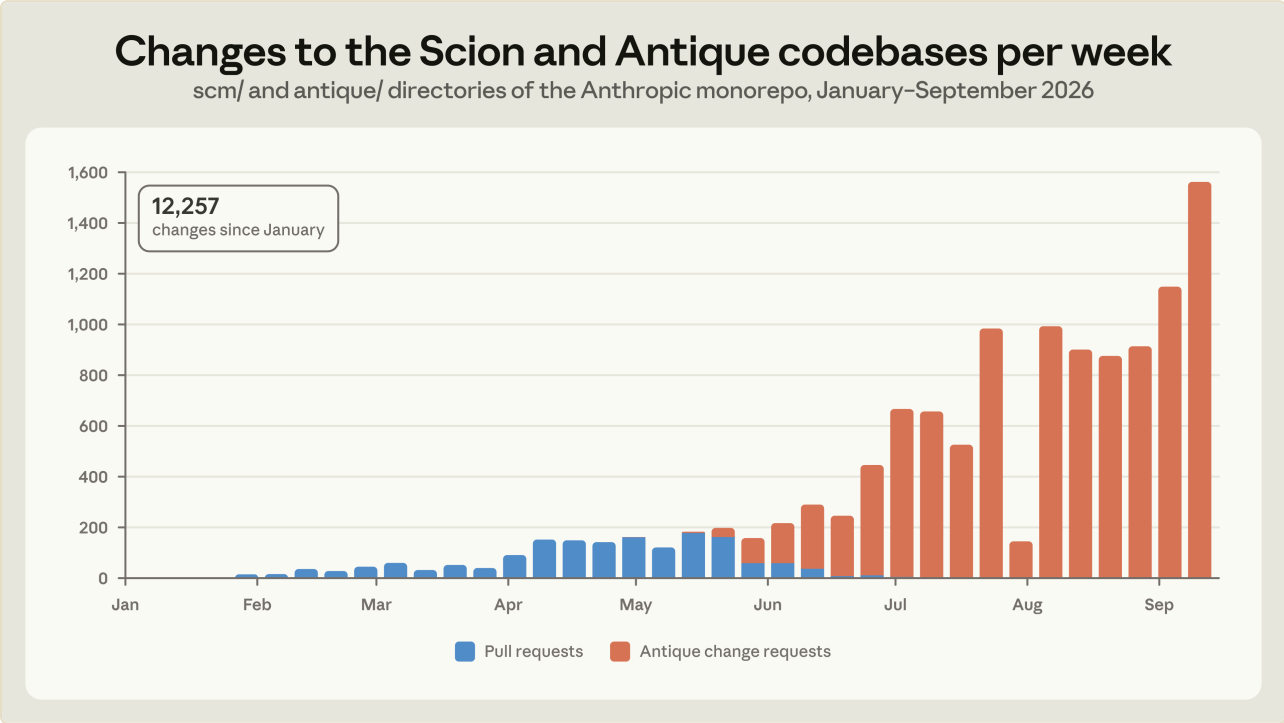
Scionのチェックアウトアーキテクチャ。

このシステムの各層は99%を超えるキャッシュヒット率を維持しており、長期間保持する状態も必要ないため、必要に応じて容易にスケールアップできます。私たちのScionインフラのフットプリントは、CPUとメモリの要件で見ると、以前のgitミラーリングインフラの200分の1になりました。CIと研究ジョブのチェックアウトは2桁速くなり（分単位ではなく秒単位になりました）、フリート全体でかなりのキャパシティが解放されました。CIだけでもCPUキャパシティが25%向上しました。クライアントもキャッシュも、リポジトリのメンテナンスを行う必要がないため、ジョブのライフサイクル管理も簡素化されました。

Scionの構築から学んだこと

私たちがScionの構築を始めたのは、Claude Mythos PreviewとClaude Tagが社内で利用可能になったのとほぼ同じ時期でした。これらのリリースはどちらも、Scionの必要性を同時に増大させると同時に、Scionをどれだけ速く構築できるかを大幅に加速させました。

特に印象的な例を一つ紹介します。この年の早い時期に、私たちはgitからScionへの取り込みパイプラインの最初のバージョンをプロトタイプとして、ほぼ手作業で書いていました。それは良い出来だと思っていました——Claude Mythos Previewの初期の社内スナップショットを実際に試すまでは。私たちはそれに全体を書き直すよう依頼しました。それ以上の指示を何も与えなかったにもかかわらず、取り込みの順序やデータ構造に関する暗黙の不変条件をすべて保持したまま、メモリ使用量が20分の1、速度が4倍になったバージョンを作り上げたのです。



2026年1月から9月までの、ScionとAntiqueのコードベースへの週あたりのマージ変更数。

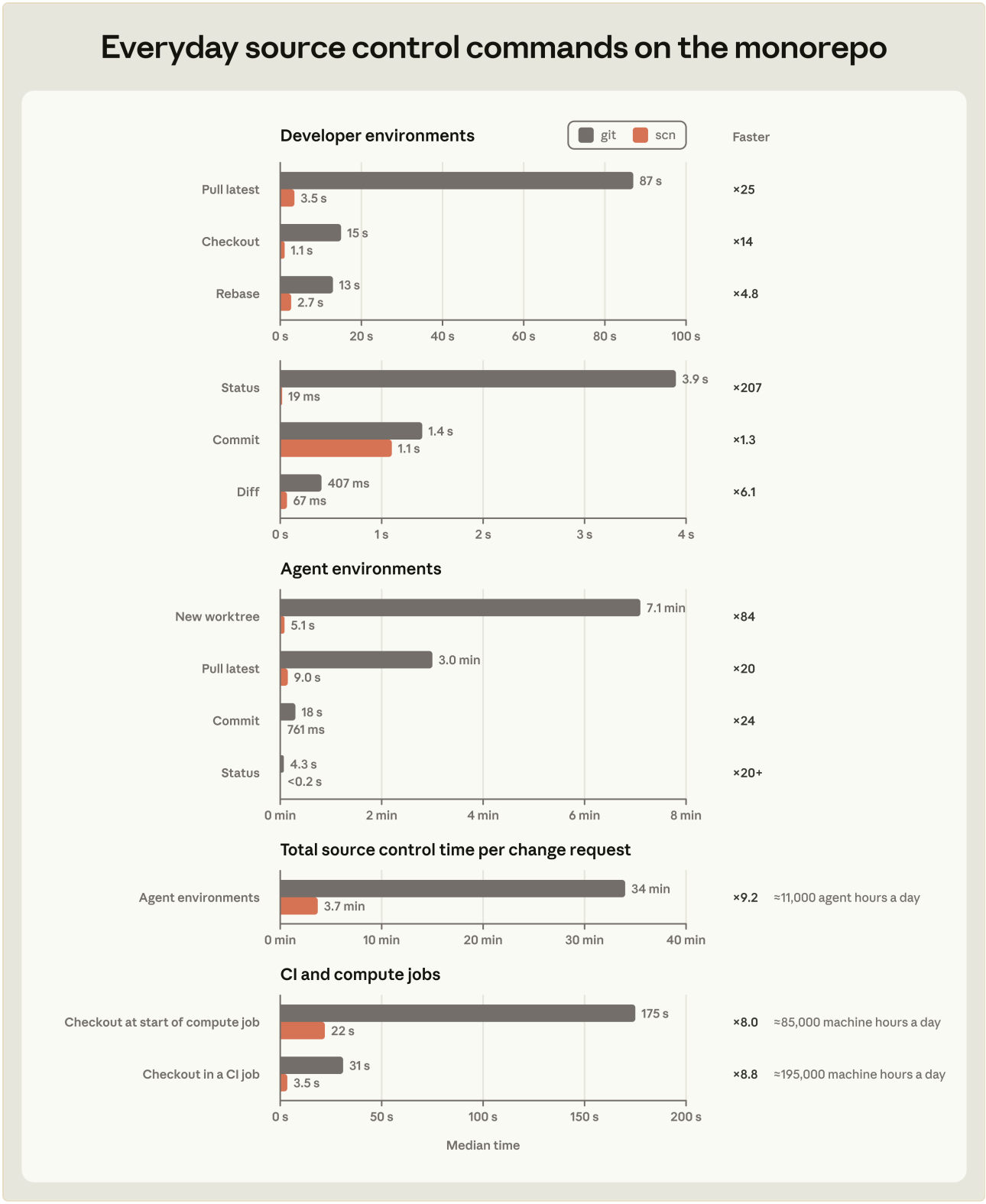
Scionを構築し、運用する中で学んだことは、エージェントスケールのソフトウェア構築がどのようなものになりうるかについて、興味深い教訓を与えてくれます。

小規模なチームによる迅速な開発が、より一般的になっていくでしょう。 わずか7人のチームが、ScionとAntique（私たちのコードレビューツールで、これだけで別のブログ記事が書けるくらいの内容があります）に加えて、半ダースほどの関連システムを構築し、5ヶ月で完全に移行しました。しかもこれを、1日あたりにマージされるコードの量が10倍に増加している最中に行ったのです。これまでであれば、このようなソースコード管理システムを開発・展開するには、企業が何年もかけて段階的に移行し、はるかに多くのエンジニアを要したはずです。私たちは、このような迅速で機動的な移行が今後より一般的になっていくと考えています。

エンジニアの価値が下がったわけではありませんが、その仕事の内容は変化しています。 このシステムに取り組んだエンジニア全員が、今では「コードを*書く*」という点ではClaudeの方が自分より優れていると評価するでしょう。しかし私たちのエンジニアは、システム間の高レベルなコンポーネントや設計レベルのトレードオフに関する、何物にも代え難い理解を依然として提供し、解決したい問題と、必要だと分かっているスケールに

ついて、情報に基づいた選択を行いました。多くの場合、エンジニアの役割はClaudeに「だめだ」と言うことでした。エージェントは、そもそも許容されるべきではないパターンを満たすために、システムの核心的な制約に違反してしまうことがあります。これは私たちの移行作業において特によく見られました。例えば、変更がmainにマージされた時期を調べるために、私たちのコードレビューツールに問い合わせるのではなく、コミットログを線形に読んでブルートフォースで調べようとするシステムが複数ありました。Anthropicはアドバーサリアルレビューも広範に活用していますが、これは多くの面で非常に効果的である一方、近視眼的になることもあります。そのため、熟練したエンジニアは、仮説上の懸念に反応してシステムの一部を過剰に設計してしまわないよう、Claudeを誘導する必要がありました。例えば、あるレビューは別の社内サービスを信頼することに対して過度に防御的で、その結果Claudeが過剰な検証と複雑さを追加してしまったことがありました。私たちのチームメンバーは、このシステムへのすべての変更リクエストを依然としてレビューしていますが、今ではすべての行を読むのではなく、その変更の*意図*を検討するようになっています。

最適化と品質を大幅に向上させることができました。 私たちが自分たちで書くよりもはるかに速いペースで、Claudeが膨大な量のコードを書く能力のおかげで、Scionは私たちがこれまでに手がけた中で最もテストが充実し、最も防御が固いコードベースの一つになっています。タイトなスケジュールの中でも、私たちはこれまで遭遇したあらゆる非自明なインフラとソースコード管理の相互作用について、ハーメティックなシミュレーションを構築することができ、ファジングや高度なレート制限も実装できました。Claudeはまた、ある種の最適化を気軽に見つけてくれました——例えば、1つのインデックスが見落とされていた祖先関係クエリに対するアルゴリズム的な最適化などです。これは、専門家であれば分析とチューニングに何日も、あるいは何週間もかかるようなものでしたが、Claudeはそれをやってのけて、さらに翌日には同じことをもう一度やってくれました。



ネイティブのScionリポジトリで観測されたパフォーマンス改善。

モノレポのセキュリティの新しい形の基盤を築きました。巨大なモノレポにおいて、安全性、セキュリティ、コンプライアンスを管理することは、規模とスケールを管理することと同じくらい重要です。私たちは最初からこれらを念頭に置いてScionを構築しました。リポジトリに対する豊富なアクセス制御に加えて、個々のディレクトリに権限を適用する機能もサポートしており、モノレポの利点を失うことなく、最もセンシティブなコードを保

護できます。さらに、私たちが構築したコードレビューツールは、承認に関する複雑なセマンティックな制約を表現できるため、コメントのみの変更のような単純なものと、センシティブなロジックの変更を区別することができ、人間のレビュアーが最も重要な変更に集中できるようになっています。

新しいレベルの運用上の優秀さが可能になりました。 私たちのチームはClaude Tagを頻繁に活用しており、これがシステムの各コンポーネントを監視する上で役立っています。例えば、私たちはSlackチャンネルを作成し、Claudeに単一のコンポーネントの健全性に強迫的なほど注意を払うという役割を与えました。私たちはこうしたClaudeを「スーパーファン」と呼んでいます。scion-daemonのスーパーファン、scion-cacheのスーパーファン、Antiqueマージキューのスーパーファンなどがいます。彼らは私たちのテレメトリとログにアクセスでき、他に何もすることがないかのようにグラフを更新し続け、何か異常がないかを探す、経験豊富なエンジニアのように振る舞います。私たちはすべてのシステムに対して従来型のアラートも用意していますが、スーパーファンはそうしたアラートが一次元的に見えるくらい、微妙で新しい問題を検知することができます。

私たちは前提を完全に見直しています。 このプロジェクトは、人間のスケール向けに設計された他のワークフローについても、非常に厳しい見直しを行うきっかけとなりました。例えば、きちんとしたマージキューは、エージェントのスケールではそれほど意味を持ちません。テストの実行中に何百、あるいは何千もの変更がマージされ得るとき、CIとは一体何を意味するのでしょうか？業界標準である、マージキューに対するグローバルなリポジトリロックを効果的に再現することは私たちにはできなかったため、代わりにマージ後・デプロイ前の自動テストに投資してきました。これは、同時に行われた変更同士がコンフリクトする可能性があるということでもありますが、Claudeは通常、人間がそのコンフリクトに気づくよりも速く修正することができます。

より広いエコシステムへの貢献

私たちはScionを、能力を増し続けるAIエージェントが引き起こすスケーリングの課題に対する答えの一部にすぎないと考えています。私たちはこの詳細を、同様の課題にチームが取り組む際の一例として共有していますが、私たちの解決策や検討事項がすべてのケースに適合するとは考えていません。

Scionは、一般用途でgitを置き換える適切な選択肢では決してありません。ここで概説した考え方は、数万人のエンジニア、あるいはコードを量産するエージェント群を抱える、本当に巨大なモノレポにのみ意味をなすものです。さらに、Anthropicは、社内利用のためにトークンを費やす能力が、他のどの企業が現実的に費やしたいと考える量をも大きく上回っているという点で特異な存在です。私たちがこれを行うのは、自分たち自身のソフトウェア開発の必要性に対応するためであると同時に、Claudeの能力をテストするためでもあります。

実用的な観点から言えば、Scionを他の組織にとって広く有用な形で提供することは難しいでしょう。gitとは大きく異なるため、企業は開発ライフサイクルのあらゆる部分を移行する必要があり、さらに独自のコードレビュー、コード検索、CIも必要になります。私たちの社内版のこれらのコンポーネントは非常にあつらえのもの

であるため、他の組織が採用するのは難しいでしょうし、すべての人を満足させようとする公開版を構築しようとしても、実際にはあまり有用なものにはならないのではないかと懸念しています。

しかし、私たちはこの種の問題が今後6〜12ヶ月のうちにより一般的になると予想しており、gitを使うすべての企業が同様の課題に単独で立ち向かうのは、エンジニアリングリソースの非効率な使い方になってしまいます。それを踏まえると、私たちにできる最も影響力のあることは、自分たちの考え方を共有し、他の開発者がScionの最も成功した部分の一部を、全体的なモデルから逸脱することなくgitに適用できるようにすることだと考えています。gitサーバーは、gitクライアントと同じように動作しなければならないわけではありません。大幅なスケーリングの改善をもたらす、新しいサーバーアーキテクチャは存在し得ます。gitプラットフォームは、Scionのような包括的なサーバーサイドRPCを提供できますし、場合によっては、gitクライアントが話せるように学習できる、フォージ間で標準化された形で提供することもできるでしょう。ネゴシエーションプロトコルも、今日のような見た目である必要はなく、gitの様々なスパース機能やシャロー機能も、うまく組み合わせられるようにできるはずです。

最後に、gitコミュニティがこのプラットフォームを数百万人の開発者のためにスケールさせるために、20年近くを費やしてきたことに敬意を表したいと思います。私たちの貢献はそれよりもずっと狭い範囲のものです。エージェントによるソフトウェア開発がこれらのシステムとどのように相互作用するかについての、私たちの早期の視点を共有し、GitHub、GitLab、そしてOSSコミュニティと連携して、開発者の変化するニーズをサポートしていくことです。ソフトウェア開発ライフサイクルがAIの指数関数的な成長と交差していく中で、私たちは皆、同じ船に乗っているのです。

謝辞

Nick G.とソースコード管理チーム（S.G.、R.K.、Dan C.、Garrett H.、Ben L.、Colin M.、Joe P.）による執筆です。この野心的な旅を通じてソースコード管理チームを支えてくれたVilas J.とSergiu D.、そしてAntiqueへの初期の貢献をしてくれたAmir O.に感謝します。

脚注

1. マージの最初の親を基準に変更ファイルを保存するのは、スタイル上の選択です。実際には、モノレポにおけるトランクベース開発では、人々はメインラインのコミットに関する履歴やblameにしか関心を持たない、ということが分かっています。